Research_HTTP-Request-smuggling

Kim Kyung Ho | Jun 2, 2021 | 0 Comments

Table of contents

1. Introduction
2. Background knowledge
2.1 HTTP request
2.2 Length structure
2.3 Connection
3. Attack
3.1 Request smuggling
3.2 Attack scenarios
3.3 Cache poisoning
3.4 Cache deception
4. Conclusion
Reference

1. Introduction

HTTP Request Smuggling attack is a technique that exploits the vulnerability of the parsing of HTTP packets by the front-end server and back-end server, and it is a technique that can cause various attacks such as bypassing the firewall, web cache poisoning, and web cache deception. This study will first explain the background knowledge necessary for understanding HTTP Request Smuggling, and then explain the attack technique and the attack scenarios that can be used in each attack technique.

2. Background knowledge

2.1 HTTP request

- Request line
  - Consists of three parts, namely the HTTP request Method, URI, and HTTP version, separated by a space.
- Header
  - Consists of a key and a value, separated by a colon (:), and ends with a newline character (\r\n).
- Blank line
  - Indicates the end of the header, and consists only of a newline character (\r\n).
- Body
  - Contains the data to be sent to the server, such as form data or file uploads.

2.2 Length structure

- Content-Length
  - Indicates the length of the body part, in bytes. For example, if the body part is 100 bytes, the value of Content-Length is 100.
- Transfer-Encoding: chunked
  - Indicates that the body part is sent in chunks. Each chunk consists of the size of the chunk and the data, and ends with a zero-length chunk. For example, if the body part is "Hello", it is sent as follows:

```
5\r\n
Hello\r\n
0\r\n
\r\n
```

2.3 Connection

- Keep-Alive
  - Indicates that the connection should be maintained after the HTTP request is processed. This allows multiple HTTP requests to be sent over the same connection, which can improve performance.
- Connection: close
  - Indicates that the connection should be closed after the HTTP request is processed. This means that a new connection must be established for each HTTP request.

3. Attack

3.1 Request smuggling

HTTP Request Smuggling attacks are a technique that exploits the vulnerability of the front-end server and back-end server to parse HTTP packets differently, causing the back-end server to interpret a single HTTP request as two or more requests. This can be used to bypass security measures, such as firewalls, or to perform other attacks, such as cache poisoning or cross-site scripting (XSS).

Normal HTTP request

HTTP Request Smuggling

HTTP Request Smuggling attacks can be performed by sending a single HTTP request that contains both the Content-Length and Transfer-Encoding: chunked headers. The front-end server will process the request based on the Content-Length header, while the back-end server will process the request based on the Transfer-Encoding: chunked header. This can cause the back-end server to interpret the request as two separate requests, allowing the attacker to bypass security measures and perform other attacks.

For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to bypass security measures and perform other attacks.

3.2 Attack scenarios

- Bypassing the firewall
  - Attackers can use HTTP Request Smuggling attacks to bypass firewalls and access internal networks or sensitive data.
  - For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to bypass the firewall and access internal networks or sensitive data.
- Cache poisoning and cache deception
  - HTTP Request Smuggling attacks can be used to perform cache poisoning and cache deception.
  - For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform cache poisoning or cache deception.
- Other attacks
  - HTTP Request Smuggling attacks can be used to perform other attacks, such as cross-site scripting (XSS) or SQL injection.
  - For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform other attacks, such as cross-site scripting (XSS) or SQL injection.

3.3 Cache poisoning

Normal HTTP request

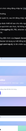
HTTP Request Smuggling

HTTP Request Smuggling attacks can be used to perform cache poisoning. For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform cache poisoning.

For example, if the attacker sends a request that contains both headers and causes the back-end server to interpret the request as two separate requests, the first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform cache poisoning. The second request will be cached by the front-end server, and subsequent requests will be served from the cache, allowing the attacker to perform other attacks, such as cross-site scripting (XSS) or SQL injection.

3.4 Cache deception

Normal HTTP request



HTTP Request Smuggling

HTTP Request Smuggling attacks can be used to perform cache deception. For example, if the front-end server processes the request based on the Content-Length header and the back-end server processes the request based on the Transfer-Encoding: chunked header, the attacker can send a request that contains both headers and cause the back-end server to interpret the request as two separate requests. The first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform cache deception.

For example, if the attacker sends a request that contains both headers and causes the back-end server to interpret the request as two separate requests, the first request will be processed by the front-end server and the second request will be processed by the back-end server, allowing the attacker to perform cache deception. The second request will be cached by the front-end server, and subsequent requests will be served from the cache, allowing the attacker to perform other attacks, such as cross-site scripting (XSS) or SQL injection.

4. Conclusion

HTTP Request Smuggling attacks are a technique that exploits the vulnerability of the parsing of HTTP packets by the front-end server and back-end server, and it is a technique that can cause various attacks such as bypassing the firewall, web cache poisoning, and web cache deception. To prevent HTTP Request Smuggling attacks, it is necessary to ensure that the front-end server and back-end server parse HTTP packets in the same way, and to use security measures such as firewalls or other security measures to prevent other attacks such as cross-site scripting (XSS) or SQL injection.

Reference

- Request Smuggling: The Ultimate Guide - Cobalt
- HTTP Request Smuggling - Portswigger
- Request Smuggling - OWASP
- Request Smuggling - Acunetix
- Request Smuggling - Imperva
- Request Smuggling - Akamai
- Request Smuggling - Cloudflare
- Request Smuggling - Fastly
- Request Smuggling - Google
- Request Smuggling - Microsoft
- Request Smuggling - Amazon
- Request Smuggling - Apache
- Request Smuggling - Nginx
- Request Smuggling - Varnish
- Request Smuggling - Squid
- Request Smuggling - HAproxy
- Request Smuggling - Tomcat
- Request Smuggling - Jetty
- Request Smuggling - Node.js
- Request Smuggling - Ruby on Rails
- Request Smuggling - Flask
- Request Smuggling - Spring
- Request Smuggling - Laravel
- Request Smuggling - Symfony
- Request Smuggling - Zend
- Request Smuggling - Drupal
- Request Smuggling - WordPress
- Request Smuggling - Joomla
- Request Smuggling - Magento
- Request Smuggling - PrestaShop
- Request Smuggling - OpenCart
- Request Smuggling - Shopify
- Request Smuggling - Wix
- Request Smuggling - Squarespace
- Request Smuggling - Weebly
- Request Smuggling - Blogger
- Request Smuggling - Tumblr
- Request Smuggling - Wordpress.com
- Request Smuggling - Medium
- Request Smuggling - Ghost
- Request Smuggling - Jekyll
- Request Smuggling - Hugo
- Request Smuggling - Gatsby
- Request Smuggling - Netlify
- Request Smuggling - Vercel
- Request Smuggling - Heroku
- Request Smuggling - AWS
- Request Smuggling - Azure
- Request Smuggling - GCP
- Request Smuggling - DigitalOcean
- Request Smuggling - Linode
- Request Smuggling - Vultr
- Request Smuggling - OVH
- Request Smuggling - Hetzner
- Request Smuggling - Scaleway
- Request Smuggling - Alibaba Cloud
- Request Smuggling - Tencent Cloud
- Request Smuggling - Baidu Cloud
- Request Smuggling - Huawei Cloud
- Request Smuggling - Kakao Cloud
- Request Smuggling - Naver Cloud
- Request Smuggling - Yandex Cloud
- Request Smuggling - Mail.ru Cloud
- Request Smuggling - Selectel
- Request Smuggling - Timeweb
- Request Smuggling - Beget
- Request Smuggling - Reg.ru
- Request Smuggling - Hostinger
- Request Smuggling - Bluehost
- Request Smuggling - HostGator
- Request Smuggling - GoDaddy
- Request Smuggling - Namecheap
- Request Smuggling - Dreamhost
- Request Smuggling - Siteground
- Request Smuggling - Liquid Web
- Request Smuggling - A2 Hosting
- Request Smuggling - InMotion Hosting
- Request Smuggling - Bluehost
- Request Smuggling - Hostinger
- Request Smuggling - Namecheap
- Request Smuggling - GoDaddy